BetaComics

Discorso di preparazione per il colloquio tecnico

---

1. Apertura: cos'e il progetto

BetaComics e un'applicazione backend di tipo e-commerce, pensata per la vendita di prodotti da collezione: fumetti, giochi da tavolo e action figure. Il progetto espone un'API REST che gestisce l'intero flusso di acquisto, dal catalogo prodotti al carrello, fino all'ordine e al checkout, con un sistema di autenticazione e autorizzazione basato sui ruoli. E scritto interamente in Java 21 e costruito su Spring Boot 4.1, con Maven come sistema di build e gestione delle dipendenze.

2. Lo stack di base

Alla base c'e Spring Boot, scelto perche permette di partire da una configurazione gia pronta grazie agli starter e alla filosofia convention over configuration: invece di configurare a mano ogni componente, si dichiarano le dipendenze e Spring fa l'auto-configurazione. Il progetto usa Java 21, quindi una versione LTS moderna.

Per ridurre il codice boilerplate e integrato Lombok: annotazioni come @Getter, @Setter, @Builder, @RequiredArgsConstructor generano automaticamente getter, setter, costruttori e il pattern builder. Questo si vede soprattutto nell'iniezione delle dipendenze: si usa la constructor injection tramite @RequiredArgsConstructor sui campi final, la pratica raccomandata da Spring perche rende le dipendenze immutabili e la classe facilmente testabile.

3. L'architettura a strati

Il progetto segue una classica architettura a livelli (layered architecture), che separa nettamente le responsabilita:

- **Controller** - gestiscono le richieste HTTP. Annotati con @RestController, mappano gli endpoint REST. Il loro unico compito e ricevere la richiesta, delegare al service e restituire una ResponseEntity.
- **Service** - contengono la logica di business. E applicato il principio di programmazione per interfacce: per ogni service c'e un'interfaccia (es. CartService) e una implementazione (CartServiceImpl). Questo disaccoppia il contratto dall'implementazione e facilita i test e futuri cambiamenti.
- **Repository** - l'accesso ai dati, realizzato con Spring Data JPA.

In mezzo c'e un livello di DTO (Data Transfer Object), diviso tra oggetti di input (le richieste, es. AddToCartRequest) e di output (le risposte, es. CartDTO). Le entita del database non vengono mai esposte direttamente verso l'esterno: si converte sempre tra entita e DTO tramite classi mapper dedicate. Questo protegge il modello dati, evita di esporre campi sensibili come la password, e da liberta di modellare la risposta come serve al frontend.

4. La persistenza dei dati

Per la persistenza si usa Spring Data JPA con Hibernate come implementazione. Il database di produzione e PostgreSQL, mentre per lo sviluppo e i test e disponibile anche H2, un database in-memory con relativa console web.

La parte di modellazione piu significativa e la gerarchia dei prodotti. C'e una classe astratta Product che contiene i campi comuni (nome, descrizione, prezzo, quantita in stock) e da questa ereditano Comic, BoardGame e ActionFigure. E stata usata la strategia di ereditarieta JOINED con una discriminator column product_type: a livello di database c'e una tabella products con i campi comuni e tabelle separate per i campi specifici di ogni tipo, collegate tramite join. E un buon compromesso tra normalizzazione e polimorfismo.

Sono state modellate anche le relazioni JPA: l'utente ha una relazione OneToOne con il carrello e OneToMany con gli ordini, gestendo correttamente il cascade e l'orphanRemoval per mantenere la coerenza. Per i prezzi si usa BigDecimal, mai double, perche e il tipo corretto per i valori monetari ed evita gli errori di arrotondamento in virgola mobile.

Una nota sulla configurazione: si usa lo schema generation di JPA per produrre gli script define.sql e drop.sql, cosi lo schema resta sempre versionato e sotto controllo.

5. Sicurezza e autenticazione

La sicurezza e gestita con Spring Security. E configurata una SecurityFilterChain personalizzata con un'autenticazione basata su form e sessione. Il flusso e: l'utente si registra, la password viene salvata cifrata con BCrypt tramite un PasswordEncoder (mai in chiaro), e al login si usa un DaoAuthenticationProvider con un CustomUserDetailsService che carica l'utente dal database.

L'autorizzazione e basata sui ruoli: ADMIN e CUSTOMER. Le regole sono granulari: la lettura del catalogo e pubblica, ma le operazioni di scrittura sui prodotti (POST, PATCH, DELETE) sono riservate all'ADMIN, cosi come il cambio di stato di un ordine. E abilitato anche @EnableMethodSecurity per proteggere i metodi a livello piu fine.

Per la gestione della sessione e impostato un limite di una sessione per utente e sono personalizzati i success/failure handler per restituire risposte JSON coerenti con il resto dell'API. Un utente corrente viene recuperato tramite un CurrentUserProvider, cosi i controller non devono ogni volta estrarre l'identita dal contesto di sicurezza.

Nota di onestà tecnica: nel codice il CSRF e attualmente disabilitato con un TO-DO. Se viene chiesto, va spiegato che e una scelta consapevole ancora da rifinire: con autenticazione a sessione andrebbe riabilitato o gestito correttamente.

6. Gestione degli errori centralizzata

E implementata una gestione delle eccezioni centralizzata con @RestControllerAdvice, nella classe ExceptionManager. Nessun controller gestisce gli errori a mano: tutte le eccezioni confluiscono qui.

E stata creata una gerarchia di eccezioni custom: BetacomicsException come classe base, da cui derivano ResourceNotFoundException e InsufficientStockException, sfruttando il polimorfismo per estrarre automaticamente lo status HTTP e il codice d'errore corretto. Le risposte d'errore sono

standardizzate in un DTO ApiErrorResponse con status, codice, messaggio, path e timestamp.

Ci sono tre livelli di gestione: le eccezioni di business, gli errori di validazione dei form (raccolti campo per campo per un feedback preciso al frontend), e una safety net generica che cattura tutto il resto. Un dettaglio importante di sicurezza: nell'handler generico non viene mai esposto il messaggio tecnico dell'eccezione al client, che potrebbe rivelare dettagli del database; si logga lo stacktrace lato server e si restituisce un messaggio generico, in linea con le best practice OWASP.

C'e anche un aspetto di internazionalizzazione: i messaggi d'errore non sono stringhe fisse ma chiavi (es. product.not.found) risolte da un MessageService, cosi i testi possono essere tradotti e centralizzati.

7. Validazione dei dati

La validazione degli input e gestita con Bean Validation (Jakarta Validation). Sui DTO di input si usano annotazioni come @NotBlank e @NotNull, e nei controller si attiva la validazione con @Valid. Sono previsti anche dei validation group per applicare regole diverse a seconda del contesto, ad esempio validare in modo diverso una creazione rispetto a un aggiornamento.

8. Documentazione dell'API

L'API e auto-documentata con Swagger / OpenAPI tramite la libreria springdoc. Questo genera automaticamente una documentazione interattiva accessibile da /swagger-ui, dove si possono vedere tutti gli endpoint e provarli direttamente. E molto comodo sia per il team frontend sia per testare rapidamente. Gli endpoint di Swagger sono esclusi dai controlli di sicurezza per restare accessibili.

9. Testing

Sul fronte dei test si usa JUnit 5 con Mockito per i test unitari e spring-boot-starter-test per i test di integrazione. E incluso anche spring-security-test, che permette di testare gli endpoint simulando utenti con ruoli diversi. I test sono organizzati in una test suite (SuiteClass) che li raggruppa, con test dedicati per i vari domini: prodotti, fumetti, giochi da tavolo, action figure e la cancellazione.

10. Chiusura

In sintesi, BetaComics e un backend REST costruito con uno stack Spring Boot moderno e ben strutturato: architettura a strati con separazione netta delle responsabilita, persistenza con JPA e una gerarchia di entita polimorfica, sicurezza a ruoli con Spring Security, gestione centralizzata di errori e validazione, documentazione automatica con OpenAPI e una copertura di test. Il tutto seguendo le buone pratiche: iniezione via costruttore, uso dei DTO, password cifrate e nessuna esposizione di dettagli tecnici al client.

Domande probabili dell'esaminatore

Domanda	Risposta chiave

Perche ereditarieta JOINED e non SINGLE_TABLE?	JOINED evita colonne nulle e mantiene il DB normalizzato; SINGLE_TABLE e piu veloce ma spreca spazio con campi nulli.
Perche DTO invece di esporre le entita?	Sicurezza (si nasconde la password), disaccoppiamento dal DB, si evitano problemi di serializzazione lazy/ricorsione.
Perche constructor injection?	Dipendenze immutabili (final), testabilita, niente dipendenze nascoste.
Perche BigDecimal per i prezzi?	double/float hanno errori di arrotondamento binari, inaccettabili sul denaro.
Come vengono cifrate le password?	BCrypt tramite PasswordEncoder; e una funzione di hash con salt, non reversibile.
CSRF disabilitato: e corretto?	E un TO-DO; con sessioni andrebbe riabilitato. Dimostra consapevolezza del problema.
Differenza @RestController vs @Controller?	@RestController = @Controller + @ResponseBody: serializza direttamente in JSON.